# Intro AI — HW2

MohammadAmin HorriFrahani - S.ID = 40216673

November 26, 2025

**Abstract**

Hopfield-network exercise

## Contents

**1 Problem statement** **2**

**2 Solution** **2**

2.1 (a) Weights . . . . . 2

2.2 (b) Fixed points . . . . . 2

2.3 (c) Recall steps and energy . . . . . 3

2.4 (d) Prototype link . . . . . 3

2.5 (e) Conclusion . . . . . 3

**A Appendix A: MATLAB code reproducing computations** **3**

**B Appendix B: numeric checks** **4**

# 1 Problem statement

Binary Hopfield network with $N = 4$. Store two patterns via Hebbian learning:

$$p^{(1)} = (+1, +1, -1, -1), \qquad p^{(2)} = (+1, -1, +1, -1).$$

Weights:

$$W = \frac{1}{N} \sum_{\mu=1}^{2} p^{(\mu)} p^{(\mu)\top}, \qquad W_{ii} = 0, \quad W = W^\top.$$

Energy:

$$E(s) = -\tfrac{1}{2} s^\top W s.$$

Asynchronous update order: $1 \to 2 \to 3 \to 4$, with

$$s_i \leftarrow \operatorname{sgn}\big((Ws)_i\big),$$

and $\operatorname{sgn}(0)$ leaves $s_i$ unchanged.

Tasks: (a) compute $W$ explicitly. (b) verify stored patterns are fixed points. (c) from initial $s^{(0)} = (+1, -1, -1, -1)$ run asynchronous updates for two full sweeps; list state after each single-neuron update and energy after each sweep. (d) starting from $x = (+1, +1, +1, -1)$ run one sweep and say which prototype basin it enters. (e) short comparison to K-means Voronoi cells.

# 2 Solution

## 2.1 (a) Weights

Compute outer-products and average (then zero the diagonal):

$$W = \frac{1}{4}\big(p^{(1)}p^{(1)\top} + p^{(2)}p^{(2)\top}\big) = \begin{pmatrix} \frac{1}{2} & 0 & 0 & -\frac{1}{2} \\ 0 & \frac{1}{2} & -\frac{1}{2} & 0 \\ 0 & -\frac{1}{2} & \frac{1}{2} & 0 \\ -\frac{1}{2} & 0 & 0 & \frac{1}{2} \end{pmatrix}.$$

Enforce $W_{ii} = 0$:

$$\boxed{W = \begin{pmatrix} 0 & 0 & 0 & -\frac{1}{2} \\ 0 & 0 & -\frac{1}{2} & 0 \\ 0 & -\frac{1}{2} & 0 & 0 \\ -\frac{1}{2} & 0 & 0 & 0 \end{pmatrix}.}$$

## 2.2 (b) Fixed points

Compute $Wp^{(1)}$ and $Wp^{(2)}$:

$$Wp^{(1)} = \begin{pmatrix} 0.5 \\ 0.5 \\ -0.5 \\ -0.5 \end{pmatrix}, \quad Wp^{(2)} = \begin{pmatrix} 0.5 \\ -0.5 \\ 0.5 \\ -0.5 \end{pmatrix}.$$

Applying $\operatorname{sgn}(\cdot)$ (with $\operatorname{sgn}(0)$ leaving bit unchanged) yields exactly $p^{(1)}$ and $p^{(2)}$. So both stored patterns are fixed points.

### 2.3 (c) Recall steps and energy

Energy formula:

$$E(s) = -\tfrac{1}{2} s^\top W s.$$

Start $s^{(0)} = (+1, -1, -1, -1)$. Update order $1 \to 2 \to 3 \to 4$. I list state *after each single-neuron update.*

**Update 1:** After updating neuron 1: $s = (+1, -1, -1, -1)$.
**Update 2:** After updating neuron 2: $s = (+1, +1, -1, -1)$.
**Update 3:** After updating neuron 3: $s = (+1, +1, -1, -1)$.
**Update 4:** After updating neuron 4: $s = (+1, +1, -1, -1)$.

Energy after sweep 1 (after 4 updates):

$$E = -1.0.$$

Second sweep (same order) leaves the state unchanged:

**Update 5:** After updating neuron 1 (sweep 2): $s = (+1, +1, -1, -1)$.
**Update 6:** After updating neuron 2: $s = (+1, +1, -1, -1)$.
**Update 7:** After updating neuron 3: $s = (+1, +1, -1, -1)$.
**Update 8:** After updating neuron 4: $s = (+1, +1, -1, -1)$.

Energy after sweep 2:

$$E = -1.0.$$

Thus after the first sweep the network reached the stored pattern $p^{(1)}$ and remained there.

### 2.4 (d) Prototype link

Start $x = (+1, +1, +1, -1)$ and run one asynchronous sweep $1 \to 2 \to 3 \to 4$:

- Updating neuron 1: no change.
- Updating neuron 2: its field is negative so $s_2$ flips to $-1$, resulting in $s = (+1, -1, +1, -1) = p^{(2)}$.
- Remaining updates keep it at $p^{(2)}$.

So $x$ enters the basin of prototype $p^{(2)}$.

### 2.5 (e) Conclusion

Hopfield attraction basins are energy basins shaped by learned correlations; they are not identical to K-means Voronoi cells (which are Euclidean-distance partitions). If prototypes overlap or are highly correlated, Hopfield networks can produce spurious attractors (mixed states) or reduced basins of attraction, harming recall.

## A Appendix A: MATLAB code reproducing computations

```
% hopfield_sim.m
% Compute weights, verify fixed points, simulate asynchronous updates.

p1 = [1; 1; -1; -1];
p2 = [1; -1; 1; -1];
N = 4;

W = (p1*p1' + p2*p2')/N;
W(1:5:end) = 0; % set diagonal to zero
```

```

disp('Weight matrix W:')
disp(W)

% Verify fixed points
wp1 = W * p1;
wp2 = W * p2;
disp('W*p1 ='); disp(wp1')
disp('sign(W*p1) ='); disp(sign(wp1)')
disp('W*p2 ='); disp(wp2')
disp('sign(W*p2) ='); disp(sign(wp2)')

% Energy function
energy = @(s) -0.5 * s' * W * s;

% (c) asynchronous recall from s0
s = [1; -1; -1; -1];
order = [1 2 3 4];
states = zeros(4,8);
k = 1;
for sweep = 1:2
  for idx = order
    h = W(idx,:) * s;
    if h > 0, s(idx) = 1;
    elseif h < 0, s(idx) = -1;
    end
    states(:,k) = s;
    k = k + 1;
  end
  fprintf('After sweep %d, state = [%s], energy = %.2f\n', sweep,
      num2str(s.'), energy(s));
end
disp('States after each single update (columns):'); disp(states)

% (d) test unseen x
x = [1;1;1;-1];
s = x;
for idx = order
  h = W(idx,:) * s;
  if h > 0, s(idx) = 1;
  elseif h < 0, s(idx) = -1;
  end
end
fprintf('After one sweep from x, state = [%s]\n', num2str(s.'));
```

Listing 1: MATLAB script: compute W, verify fixed points, simulate asynchronous updates

# B Appendix B: numeric checks

Energy at the final attractor $p^{(1)}$: $E = -1.0$ (numeric).

### Question 2: SOM

The output weights are

| Node | $A$ | $B$ | $C$ | $D$ | $E$ | $F$ |
|---|---|---|---|---|---|---|
| $w_1$ | $-1$ | 0 | 3 | $-2$ | 3 | 4 |
| $w_2$ | 2 | 4 | $-2$ | $-3$ | 2 | $-1$ |

Input: $x = (2, -4)$. Distances (squared) and steps:

$$\begin{aligned}
d_A^2 &= (2-(-1))^2 + (-4-2)^2 = 3^2 + (-6)^2 = 9 + 36 = 45,\\
d_B^2 &= (2-0)^2 + (-4-4)^2 = 2^2 + (-8)^2 = 4 + 64 = 68,\\
d_C^2 &= (2-3)^2 + (-4-(-2))^2 = (-1)^2 + (-2)^2 = 1 + 4 = 5,\\
d_D^2 &= (2-(-2))^2 + (-4-(-3))^2 = 4^2 + (-1)^2 = 16 + 1 = 17,\\
d_E^2 &= (2-3)^2 + (-4-2)^2 = (-1)^2 + (-6)^2 = 1 + 36 = 37,\\
d_F^2 &= (2-4)^2 + (-4-(-1))^2 = (-2)^2 + (-3)^2 = 4 + 9 = 13.
\end{aligned}$$

Minimum squared distance is $d_C^2 = 5 \Rightarrow$ **winner** $=$ **C**.

Adaptation rule: $w' = w + \alpha h(x - w)$ with $\alpha = 0.5$. Neighbourhood $h = 1$ for winner, $h = 0.5$ for immediate neighbours (B and D), $h = 0$ otherwise.

Compute updates:

$$\begin{aligned}
w'_C &= (3,-2) + 0.5 \cdot 1 \cdot ((2,-4) - (3,-2)) = (3,-2) + 0.5 \cdot (-1,-2) = (2.5, -3.0),\\
w'_B &= (0,4) + 0.5 \cdot 0.5 \cdot ((2,-4) - (0,4)) = (0,4) + 0.25 \cdot (2,-8) = (0.5, 2.0),\\
w'_D &= (-2,-3) + 0.25 \cdot ((2,-4) - (-2,-3)) = (-2,-3) + 0.25 \cdot (4,-1) = (-1.0, -3.25).
\end{aligned}$$

Nodes $A, E, F$ unchanged. Thus updated weights:

| Node | $A$ | $B$ | $C$ | $D$ | $E$ | $F$ |
|---|---|---|---|---|---|---|
| $w'_1$ | $-1$ | 0.5 | 2.5 | $-1$ | 3 | 4 |
| $w'_2$ | 2 | 2.0 | $-3.0$ | $-3.25$ | 2 | $-1$ |

**Comparison (brief).** Feed-forward networks are usually trained supervised (minimize global loss via backprop); SOMs are unsupervised, use competitive learning and neighbourhood updates, and produce topology-preserving low-dimensional maps for clustering/visualization.

```


% som_update.m
% Determine winner for x and perform neighbourhood weight updates.

% initial weights
W = [ -1,  0,  3, -2,  3,  4 ;   % row 1 = w1
       2,  4, -2, -3,  2, -1 ]; % row 2 = w2

x = [2; -4];

% compute squared distances
d2 = zeros(1,6);
for j=1:6
    d2(j) = (x(1)-W(1,j))^2 + (x(2)-W(2,j))^2;
end
disp('Squared distances to nodes A..F:')
disp(d2)

% find winner
[~, winner] = min(d2); % index 1..6 corresponds to A..F
nodes = 'ABCDEF';
fprintf('Winner: node %c\n', nodes(winner));

% neighbourhood: ring A-B-C-D-E-F-A
% immediate neighbours of C (index 3) are 2 and 4
alpha = 0.5;
```

```
h = zeros(1,6);
h(winner) = 1;
% set immediate neighbours (modular ring)
left = mod(winner-2,6)+1;
right = mod(winner,6)+1;
h(left) = 0.5;
h(right) = 0.5;

% apply updates
Wnew = W;
for j=1:6
    if h(j) ~= 0
        Wnew(:,j) = W(:,j) + alpha * h(j) * (x - W(:,j));
    end
end

disp('Updated weights (columns A..F):')
disp(Wnew)
```

Listing 2: MATLAB script